

PS1

ChronoSolve

Adaptive Constraint-Optimized Academic Scheduling

Generate once. Adapt intelligently. Disrupt minimally.

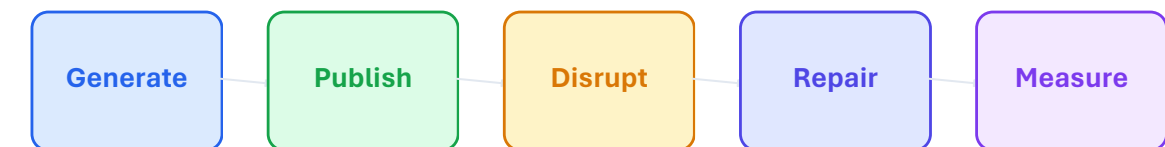
Team Details:

Team Name : 4L's

Team Members:

- Pruthvi Avhad
- Siddhesh Atugade
- Vedant Aundhakar
- Atharva Gramopadhye

From static timetable to adaptive repair



DBMS	OS	BREAK	CN	
LAB		AI		SE
	DAA		LAB	
ML		CN		MOVED

Proposed Solution

Not a static generator — an adaptive repair engine for real academic disruptions.

UNDERLYING PROBLEM

A timetable does not stay static. Faculty leave, room/lab unavailability, events and policy changes can suddenly invalidate an already-published schedule.

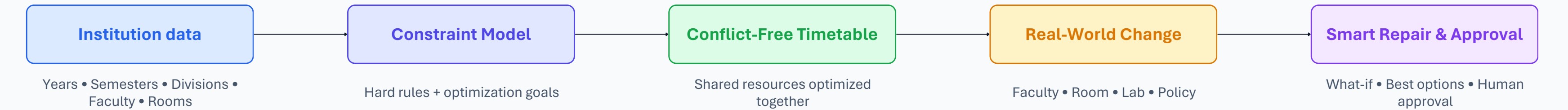
WHAT WE BUILT

One constraint-optimization system that schedules years, semesters and divisions together, then intelligently repairs the published timetable when real-world constraints change.

PROOF

Every proposed repair is previewed first. Faculty can request changes, ChronoSolve recommends valid alternatives, and only the coordinator can approve and publish the revision.

Work Flow



Innovation & uniqueness

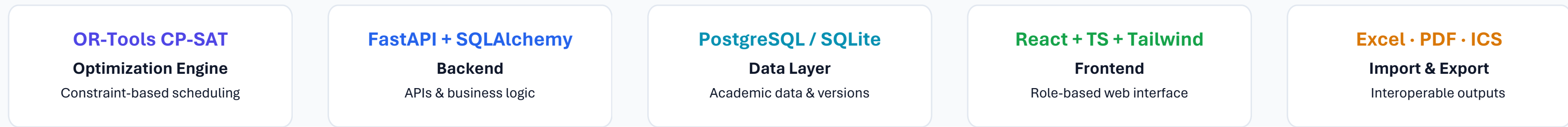
Dimension	Typical solution	ChronoSolve
After a disruption	Rebuild or manually patch the timetable	Repairs only what actually needs to change
Scheduling scope	Timetables often handled in isolated parts	Years, divisions, faculty, rooms and labs optimized together
Decision making	Manual or first-valid alternative	Best feasible option selected using constraint optimization
Human control & transparency	Changes may be difficult to review or justify	Preview, explain, approve and then publish

The ChronoSolve Difference

- Preserve — protect unaffected sessions
- Optimize — choose the best feasible repair
- Explain — show why a session moved
- Approve — human remains in control

Technical Approach

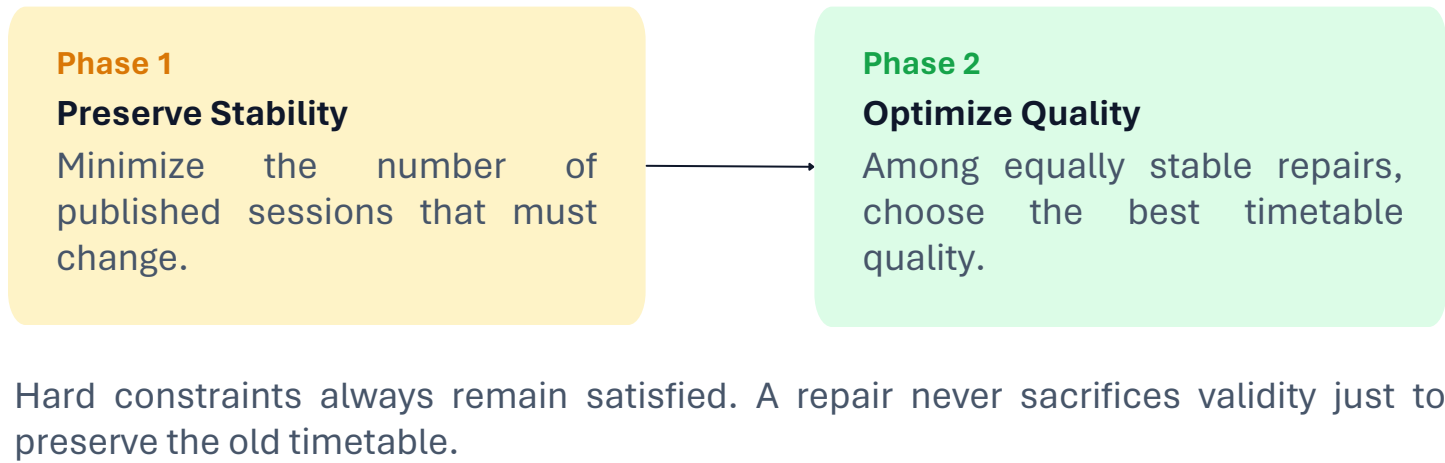
A formal constraint model, not a greedy timetable filler.



How ChronoSolve Optimizes

- 1 Build legal possibilities**
Each session is matched with feasible timeslots + rooms.
- 2 Enforce hard constraints**
Faculty, batch and room clashes · availability · room/lab requirements · workload · electives · locked sessions.
- 3 Optimize timetable quality**
Reduce student gaps, faculty idle time, workload imbalance and poor room allocation.
- 4 Independently validate**
The generated timetable is checked again before it can be published.

Minimum-Disruption Repair



System process flow



Feasibility & Viability

Demonstrated with a working, tested, zero-cost prototype.

TECHNICALLY FEASIBLE
Proven Solver Stack
CP-SAT handles complex academic constraints with deterministic validation.

OPERATIONALLY FEASIBLE
Fits Existing Workflow
Import data → Generate → Review → Publish → Repair when constraints change.

DEPLOYMENT READY
Web + Local Fallback
Runs through a browser, with a local deployment option for reliability.

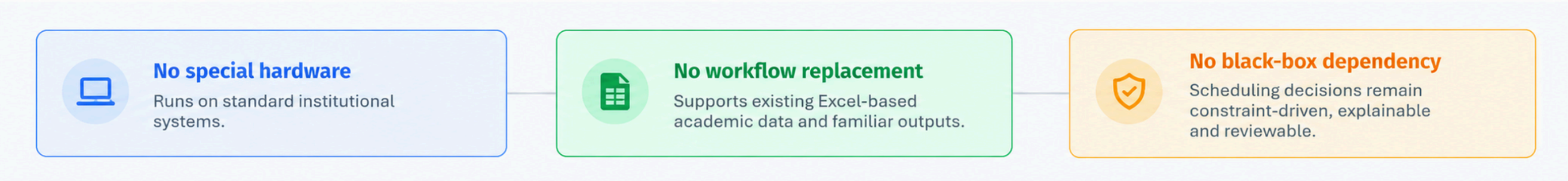
LOW-COST ADOPTION
Open-Source Core
No mandatory paid solver or AI service required for core scheduling.

Challenges handled in the product

Challenge	Mitigation
Conflicting scheduling constraints	CP-SAT enforces all hard constraints and validates feasibility before publishing
Unexpected faculty / room / lab disruption	What-if analysis and minimum-disruption re-optimization
Lack of trust in automated changes	Explainable repair, preview, human approval and version tracking
Incorrect or incomplete academic data	Input validation before data enters the scheduling engine

Why It Is Viable

- 1**Modular Architecture**
Frontend, backend, database and solver remain independent and maintainable.
- 2**Institution Configurable**
Years, semesters, divisions, faculty, rooms, labs and constraints are data-driven rather than hard-coded.
- 3**Human-Controlled**
ChronoSolve proposes; coordinators review and publish.
- 4**Scalable by Scope**
Additional years/divisions can be introduced while shared resources remain part of the same scheduling model.



Impact & Benefits

Measured on our system — not projected estimates.

Operational Impact



Schedule Stability

Preserves unaffected classes when disruptions occur.



Faster Recovery

Turns timetable repair from manual trial-and-error into an optimized workflow.



Conflict-Safe Changes

Every approved repair is revalidated against faculty, batch, room and lab constraints.



Better Resource Use

Matches sessions with suitable rooms and laboratories based on capacity and requirements.

Why Minimum - Disruption matters



Who Benefits



Research & References

The repair problem has a formal name: Minimal Perturbation Problem in course timetabling.

Reference	Research / Source	Relevance to ChronoSolve
Müller, Rudová & Barták (2005)	<i>Minimal Perturbation Problem in Course Timetabling</i>	Basis for minimum-disruption repair and preserving an existing timetable after changes
Schaerf (1999)	<i>A Survey of Automated Timetabling</i>	Foundation for constraint-based academic timetable generation
Burke & Petrovic (2002)	<i>Recent Research Directions in Automated Timetabling</i>	Supports optimization objectives, scheduling quality and practical constraints
Google OR-Tools	<i>CP-SAT Solver Documentation</i>	Technical foundation for constraint modelling, feasibility and optimization
NEP 2020 — Ministry of Education, Government of India	<i>National Education Policy 2020</i>	Supports flexible curricula, electives and multidisciplinary academic structures